

Ex. No: 4 CPU Scheduling Algorithms

1. First Come First Serve (FCFS)

C PROGRAM:

```
#include <stdio.h>

int main()
{
    int n, i;

    int bt[20], wt[20], tat[20];

    float avg_wt = 0, avg_tat = 0;

    printf("Enter Number of Processes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++)
    {
        printf("Enter Burst Time for P%d: ", i + 1);
        scanf("%d", &bt[i]);
    }

    wt[0] = 0;

    for (i = 1; i < n; i++)
    {
        wt[i] = wt[i - 1] + bt[i - 1];
    }

    for (i = 0; i < n; i++)
    {
        tat[i] = wt[i] + bt[i];

        avg_wt += wt[i];
```

```

    avg_tat += tat[i];
}

printf("\nProcess\tBT\tWT\tTAT\n");

for (i = 0; i < n; i++)
{
    printf("P%d\t%d\t%d\t%d\n", i + 1, bt[i], wt[i], tat[i]);
}

printf("\nAverage Waiting Time = %.2f", avg_wt / n);
printf("\nAverage Turnaround Time = %.2f\n", avg_tat / n);

return 0;
}

```

Output:

```

(kali㉿kali)-[~]
$ nano fcfs.c

(kali㉿kali)-[~]
$ gcc fcfs.c -o fcfs

(kali㉿kali)-[~]
$ ./fcfs
Enter Number of Processes: 2
Enter Burst Time for P1: 3
Enter Burst Time for P2: 4

Process BT      WT      TAT
P1      3        0        3
P2      4        3        7

Average Waiting Time = 1.50
Average Turnaround Time = 5.00

```

Shell Program (fcfs.sh)

```
#!/bin/bash
```

```
echo "Enter Number of Processes"
```

```
read n
```

```

for ((i=0;i<n;i++))
do
echo "Enter Burst Time for P$(i+1)"
read bt[$i]
done
wt[0]=0
for ((i=1;i<n;i++))
do
wt[$i]=$((wt[i-1]+bt[i-1]))
done
echo
echo -e "Process\tBT\tWT\tTAT"
total_wt=0
total_tat=0
for ((i=0;i<n;i++))
do
tat[$i]=$((wt[i]+bt[i]))
total_wt=$((total_wt+wt[i]))
total_tat=$((total_tat+tat[i]))
echo -e "P$(i+1)\t${bt[i]}\t${wt[i]}\t${tat[i]}"
done
avg_wt=$(awk "BEGIN {printf \"%.2f\", $total_wt/$n}")
avg_tat=$(awk "BEGIN {printf \"%.2f\", $total_tat/$n}")
echo
echo "Average Waiting Time = $avg_wt"
echo "Average Turnaround Time = $avg_tat"

```

Output:

```
(kali㉿kali)-[~]
$ nano lab4-1.sh

(kali㉿kali)-[~]
$ chmod +x lab4-1.sh

(kali㉿kali)-[~]
$ bash lab4-1.sh
Enter Number of Processes
2
Enter Burst Time for P1
3
Enter Burst Time for P2
4

Process BT      WT      TAT
P1       3       0       3
P2       4       3       7

Average Waiting Time = 1.50
Average Turnaround Time = 5.00
```

2. Shortest Job First (SJF)

C Program:

C Program

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
int n,i,j,temp;
```

```
int bt[20],wt[20],tat[20];
```

```
float avg_wt=0, avg_tat=0;
```

```
printf("Enter Number of Processes: ");
```

```
scanf("%d",&n);
```

```
for(i=0;i<n;i++)
```

```
{
```

```

printf("Enter Burst Time for P%d: ",i+1);
scanf("%d",&bt[i]);
}
// Sorting burst times in ascending order
for(i=0;i<n-1;i++)
{
for(j=i+1;j<n;j++)
{
if(bt[i] > bt[j])
{
temp = bt[i];
bt[i] = bt[j];
bt[j] = temp;
}
}
wt[0] = 0;
for(i=1;i<n;i++)
wt[i] = wt[i-1] + bt[i-1];
printf("\nProcess\tBT\tWT\tTAT\n");
for(i=0;i<n;i++)
{
tat[i] = wt[i] + bt[i];
avg_wt += wt[i];
avg_tat += tat[i];
printf("P%d\t%d\t%d\t%d\n",i+1,bt[i],wt[i],tat[i]);
}

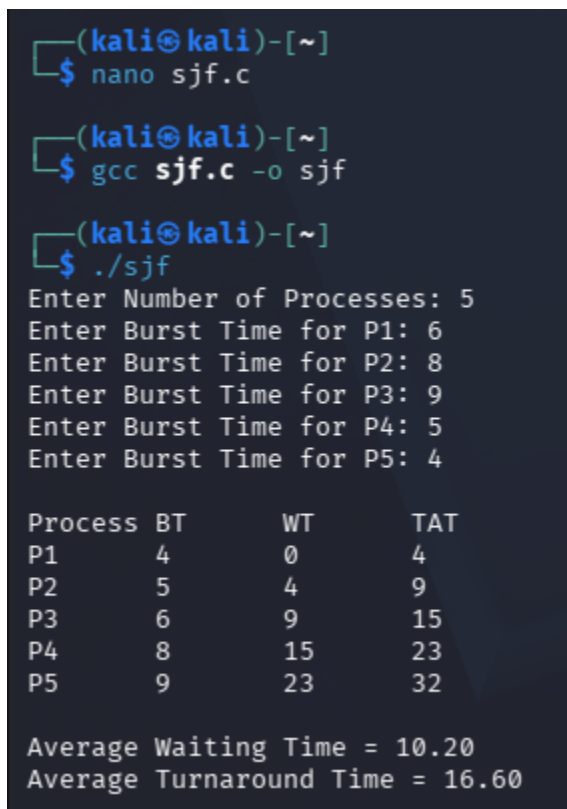
```

```

avg_wt = avg_wt / n;
avg_tat = avg_tat / n;
printf("\nAverage Waiting Time = %.2f", avg_wt);
printf("\nAverage Turnaround Time = %.2f\n", avg_tat);
return 0;
}

```

Output:



```

(kali㉿kali)-[~]
$ nano sjf.c

(kali㉿kali)-[~]
$ gcc sjf.c -o sjf

(kali㉿kali)-[~]
$ ./sjf
Enter Number of Processes: 5
Enter Burst Time for P1: 6
Enter Burst Time for P2: 8
Enter Burst Time for P3: 9
Enter Burst Time for P4: 5
Enter Burst Time for P5: 4

Process BT      WT      TAT
P1      4       0       4
P2      5       4       9
P3      6       9      15
P4      8      15      23
P5      9      23      32

Average Waiting Time = 10.20
Average Turnaround Time = 16.60

```

Shell Program (sjf.sh)

```

#!/bin/bash

echo "Enter Number of Processes"
read n

for ((i=0;i<n;i++))

```

```

do
echo "Enter Burst Time for P$((i+1))"
read bt[$i]
done

# Sorting burst times (Ascending Order)
for ((i=0;i<n-1;i++))
do
for ((j=i+1;j<n;j++))
do
if [ ${bt[i]} -gt ${bt[j]} ]
then
temp=${bt[i]}
bt[$i]=${bt[j]}
bt[$j]=$temp
fi
done
done

wt[0]=0

echo

echo -e "Process\tBT\tWT\tTAT"

total_wt=0

total_tat=0

for ((i=0;i<n;i++))
do
if [ $i -ne 0 ]
then

```

```

wt[$i]=$((wt[i-1]+bt[i-1]))
fi
tat[$i]=$((wt[i]+bt[i]))
total_wt=$((total_wt+wt[i]))
total_tat=$((total_tat+tat[i]))
echo -e "P$((i+1))\t${bt[i]}\t${wt[i]}\t${tat[i]}"
done

avg_wt=$(awk "BEGIN {printf \"%.2f\", $total_wt/$n}")
avg_tat=$(awk "BEGIN {printf \"%.2f\", $total_tat/$n}")

echo

echo "Average Waiting Time = $avg_wt"

echo "Average Turnaround Time = $avg_tat"

```

```

(kali㉿kali)-[~]
$ nano lab4-2.sh

(kali㉿kali)-[~]
$ chmod +x lab4-2.sh

(kali㉿kali)-[~]
$ bash lab4-2.sh
Enter Number of Processes
4
Enter Burst Time for P1
5
Enter Burst Time for P2
6
Enter Burst Time for P3
7
Enter Burst Time for P4
8

Process BT      WT      TAT
P1      5        0        5
P2      6        5       11
P3      7       11       18
P4      8       18       26

Average Waiting Time = 8.50
Average Turnaround Time = 15.00

```

Output:

3. Priority Scheduling

C Program:

```
#include <stdio.h>

int main()
{
    int n, i, j, temp;

    int bt[20], pr[20], wt[20], tat[20];

    float avg_wt = 0, avg_tat = 0;

    printf("Enter Number of Processes: ");

    scanf("%d", &n);

    for (i = 0; i < n; i++)
    {
        printf("\nEnter Burst Time for P%d: ", i + 1);

        scanf("%d", &bt[i]);

        printf("Enter Priority for P%d: ", i + 1);

        scanf("%d", &pr[i]);
    }

    /* Sort according to priority */

    for (i = 0; i < n - 1; i++)
    {
        for (j = i + 1; j < n; j++)
        {
            if (pr[i] > pr[j])
            {
                temp = pr[i];
                pr[i] = pr[j];
                pr[j] = temp;
            }
        }
    }

    for (i = 0; i < n; i++)
    {
        wt[i] = 0;
        tat[i] = 0;
    }

    for (i = 0; i < n; i++)
    {
        wt[i] = wt[i] + bt[i];
        tat[i] = tat[i] + wt[i];
    }

    avg_wt = (wt[0] + wt[1] + wt[2] + wt[3] + wt[4] + wt[5] + wt[6] + wt[7] + wt[8] + wt[9] + wt[10] + wt[11] + wt[12] + wt[13] + wt[14] + wt[15] + wt[16] + wt[17] + wt[18] + wt[19]) / n;
    avg_tat = (tat[0] + tat[1] + tat[2] + tat[3] + tat[4] + tat[5] + tat[6] + tat[7] + tat[8] + tat[9] + tat[10] + tat[11] + tat[12] + tat[13] + tat[14] + tat[15] + tat[16] + tat[17] + tat[18] + tat[19]) / n;

    printf("\nAverage Waiting Time = %f", avg_wt);
    printf("\nAverage Turnaround Time = %f", avg_tat);
}
```

```

        pr[j] = temp;
        temp = bt[i];
        bt[i] = bt[j];
        bt[j] = temp;
    }
}
}

/* Calculate Waiting Time */
wt[0] = 0;
for (i = 1; i < n; i++)
{
    wt[i] = wt[i - 1] + bt[i - 1];
}

printf("\nProcess\tPriority\tBT\tWT\tTAT\n");

/* Calculate Turnaround Time */
for (i = 0; i < n; i++)
{
    tat[i] = wt[i] + bt[i];
    avg_wt += wt[i];
    avg_tat += tat[i];
    printf("P%d\t%d\t%d\t%d\t%d\n",
        i + 1, pr[i], bt[i], wt[i], tat[i]);
}

avg_wt /= n;
avg_tat /= n;

printf("\nAverage Waiting Time = %.2f", avg_wt);

```

```

printf("\nAverage Turnaround Time = %.2f\n", avg_tat);

return 0;

}

```

Output:

```

(kali㉿kali)-[~]
$ nano priority.c

(kali㉿kali)-[~]
$ gcc priority.c -o priority

(kali㉿kali)-[~]
$ ./priority
Enter Number of Processes: 2

Enter Burst Time for P1: 3
Enter Priority for P1: 4

Enter Burst Time for P2: 5
Enter Priority for P2: 6

Process Priority      BT      WT      TAT
P1        4           3        0        3
P2        6           5        3        8

Average Waiting Time = 1.50
Average Turnaround Time = 5.50

```

Shell Script:

```

#!/bin/bash

echo "Enter Number of Processes"

read n

for ((i=0; i<n; i++))
do
    echo "Enter Burst Time for P$((i+1))"

    read bt[$i]

```

```

    echo "Enter Priority for P${(i+1)}"
    read pr[$i]
done

# Sort according to Priority
for ((i=0; i<n-1; i++))
do
    for ((j=i+1; j<n; j++))
    do
        if [ ${pr[i]} -gt ${pr[j]} ]
        then
            temp=${pr[i]}
            pr[$i]=${pr[j]}
            pr[$j]=$temp
            temp=${bt[i]}
            bt[$i]=${bt[j]}
            bt[$j]=$temp
        fi
    done
done

# Calculate Waiting Time
wt[0]=0

echo

echo -e "Process\tPriority\tBT\tWT\tTAT"

total_wt=0

total_tat=0

```

```

for ((i=0; i<n; i++))
do
    if [ $i -ne 0 ]
    then
        wt[$i]=$((wt[i-1] + bt[i-1]))
    fi
    tat[$i]=$((wt[i] + bt[i]))
    total_wt=$((total_wt + wt[i]))
    total_tat=$((total_tat + tat[i]))
    echo -e "P$((i+1))\t${pr[i]}\t${bt[i]}\t${wt[i]}\t${tat[i]}"
done
avg_wt=$(awk "BEGIN {printf \"%.2f\\n\", $total_wt/$n}")
avg_tat=$(awk "BEGIN {printf \"%.2f\\n\", $total_tat/$n}")
echo
echo "Average Waiting Time = $avg_wt"
echo "Average Turnaround Time = $avg_tat"

```

Output:

```
(kali㉿kali)-[~]
$ nano lab4-3.sh

(kali㉿kali)-[~]
$ chmod +x lab4-3.sh

(kali㉿kali)-[~]
$ bash lab4-3.sh
Enter Number of Processes
2
Enter Burst Time for P1
3
Enter Priority for P1
4
Enter Burst Time for P2
5
Enter Priority for P2
6

Process Priority      BT      WT      TAT
P1         4          3        0        3
P2         6          5        3        8

Average Waiting Time = 1.50
Average Turnaround Time = 5.50
```

4. Round Robin Scheduling

C Program:

```
#include <stdio.h>

int main()
{
    int n, tq, i;

    int bt[20], rem_bt[20];

    int wt[20] = {0}, tat[20];

    int time = 0, done;

    float avg_wt = 0, avg_tat = 0;

    printf("Enter Number of Processes: ");

    scanf("%d", &n);

    for (i = 0; i < n; i++)
```

```
{  
    printf("Enter Burst Time for P%d: ", i + 1);  
    scanf("%d", &bt[i]);  
    rem_bt[i] = bt[i];  
}
```

```
printf("Enter Time Quantum: ");  
scanf("%d", &tq);
```

```
do
```

```
{  
    done = 1;  
    for (i = 0; i < n; i++)  
    {  
        if (rem_bt[i] > 0)  
        {  
            done = 0;  
            if (rem_bt[i] > tq)  
            {  
                time += tq;  
                rem_bt[i] -= tq;  
            }  
            else  
            {  
                time += rem_bt[i];  
                wt[i] = time - bt[i];  
                rem_bt[i] = 0;  
            }  
        }  
    }  
}
```

```

        }
    }
}
} while (!done);
printf("\nProcess\tBT\tWT\tTAT\n");
for (i = 0; i < n; i++)
{
    tat[i] = bt[i] + wt[i];
    avg_wt += wt[i];
    avg_tat += tat[i];
    printf("P%d\t%d\t%d\t%d\n",
        i + 1, bt[i], wt[i], tat[i]);
}
avg_wt /= n;
avg_tat /= n;
printf("\nAverage Waiting Time = %.2f", avg_wt);
printf("\nAverage Turnaround Time = %.2f\n", avg_tat);
return 0;
}

```

Output:


```

(kali㉿kali)-[~]
$ nano roundrobin.c

(kali㉿kali)-[~]
$ gcc roundrobin.c -o roundrobin

(kali㉿kali)-[~]
$ ./roundrobin
Enter Number of Processes: 5
Enter Burst Time for P1: 6
Enter Burst Time for P2: 7
Enter Burst Time for P3: 8
Enter Burst Time for P4: 9
Enter Burst Time for P5: 3
Enter Time Quantum: 5

Process BT      WT      TAT
P1      6      18      24
P2      7      19      26
P3      8      21      29
P4      9      24      33
P5      3      20      23

Average Waiting Time = 20.40
Average Turnaround Time = 27.00

```

Shell Program:

```
#!/bin/bash
```

```
echo "Enter Number of Processes"
```

```
read n
```

```
for ((i=0; i<n; i++))
```

```
do
```

```
    echo "Enter Burst Time for P$((i+1))"
```

```
    read bt[$i]
```

```
    rem_bt[$i]=$(bt[$i])
```

```
done
```

```
echo "Enter Time Quantum"

read tq

time=0

while true
do
    done=1

    for ((i=0; i<n; i++))
    do
        if [ ${rem_bt[i]} -gt 0 ]
        then
            done=0

            if [ ${rem_bt[i]} -gt $tq ]
            then
                time=$((time + tq))
                rem_bt[i]=$((rem_bt[i] - tq))
            else
                time=$((time + rem_bt[i]))
                wt[i]=$((time - bt[i]))
                rem_bt[i]=0
            fi
        fi
    done
done
```

```
if [ $done -eq 1 ]
then
    break
fi
done

echo
echo -e "Process\tBT\tWT\tTAT"

total_wt=0
total_tat=0

for ((i=0; i<n; i++))
do
    tat[$i]=$((bt[i] + wt[i]))

    total_wt=$((total_wt + wt[i]))
    total_tat=$((total_tat + tat[i]))

    echo -e "P$((i+1))\t${bt[i]}\t${wt[i]}\t${tat[i]}"
done

avg_wt=$(awk "BEGIN {printf \"%.2f\", $total_wt/$n}")
avg_tat=$(awk "BEGIN {printf \"%.2f\", $total_tat/$n}")
```

echo

echo "Average Waiting Time = \$avg_wt"

echo "Average Turnaround Time = \$avg_tat"

Output:

```
(kali㉿kali)-[~]  
$ nano lab4-4.sh  
  
(kali㉿kali)-[~]  
$ chmod +x lab4-4.sh  
  
(kali㉿kali)-[~]  
$ bash lab4-4.sh  
Enter Number of Processes  
2  
Enter Burst Time for P1  
3  
Enter Burst Time for P2  
4  
Enter Time Quantum  
5  
  
Process BT      WT      TAT  
P1      3      0      3  
P2      4      3      7  
  
Average Waiting Time = 1.50  
Average Turnaround Time = 5.00
```